
EXPLORATORY DATA ANALYSIS WITH EPC INDUSTRY DATA

Kiran Kumar K Shastry

5-Year Integrated M.Sc. Economics (2024-2029)

Dr. B. R. Ambedkar School of Economics University,
Bengaluru

Period of Internship: 18th May 2026 – 14th June 2026

Report submitted to: IDEAS – Institute of Data
Engineering, Analytics and Science Foundation, ISI
Kolkata

1. Abstract

This project is Exploratory Data Analysis (EDA) with EPC (Engineering, Procurement and Construction) industry data. The dataset was analyzed using Python tools like Pandas, NumPy, and Scikit-learn. First there was the usual part, loading the data and then exploring it. Later, it included duplicate rows, dealing with missing values, putting out statistical summaries, and then doing a correlation check, just to see what moved together.

For outliers, Interquartile Range method is used, the IQR approach. There was also group based aggregation, plus feature creation and some ranking analysis, along with conditional filtering. Then, in addition to the EPC part, the MNIST digits dataset was brought in too. That one was used for K-Means clustering and checked how well the clusters worked by using the F1 score.

Overall, the work gave practical exposure to data preprocessing, exploratory analysis, making features, clustering, and then evaluating performance. In the end it helped build key basic skills for data analytics, and machine learning, mostly using Python.

2. Introduction

Engineering, Procurement, and Construction (EPC) projects start out as if they were just execution plans but then they gradually turn into data generators, pumping out large amounts of info on project budgets, costs, overall efficiency, and the daily operational performance, almost as a routine by itself. As soon as organizations start digging into this information, they usually can spot project trends, find those recurring patterns, and make better decisions. Exploratory Data Analysis (EDA) is one of the important analytical works because it shows how the data is organized, and check whether it is clean enough before jumping into more serious analysis.

For this project, Python was used to process EPC industry data. At the beginning, the workflow mostly started with loading and exploring the dataset, then checking for duplicates, plus scanning for missing entries, so in the end it doesn't end up with strange anomalies lurking around. After that, statistical summaries were created, and correlation analysis was done to understand the key features and how they connect. On top of that, outlier detection was carried out using the Interquartile Range (IQR) approach, in order to catch unusual observations that might mess up the later steps.

A few other techniques were also used, like group aggregation, feature engineering, ranking based analysis, and conditional filtering. Overall, these approaches made it easier to pull useful insights from the dataset, while providing practical experience when the data is messy, scattered, or just plain flawed in places.

So, the project had this machine learning bit, where the MNIST digits data set is used. The K-Means clustering is used to sort similar digit images into clusters, and checked the clustering quality using the F1 score. Also, it was a decent chance to get hands on with unsupervised learning, and to really understand how model evaluation plays out in normal everyday cases.

By the end, the whole thing was built in Python with support from things like Pandas, NumPy, and Scikit-learn. The work built a practical awareness across data preprocessing, exploratory analysis, reshaping raw inputs into usable features, the whole clustering process, and finally performance evaluation too.

Topics Covered During Internship Training:

During the internship, training sessions were conducted on the following topics:

- Introduction to Data Science
 - Python Programming Fundamentals
 - Data Analysis using Python
 - Machine Learning Fundamentals
 - Machine Learning Laboratory Sessions
 - LLM Fundamentals
 - Skills in communication
-

3. Objectives

- The goal is to do Exploratory data analysis on EPC industry data with Python.
 - We need to deal with duplicate records, missing values, and those outliers that show up out of nowhere.
 - Then we create statistical summaries.
 - After that, we analyze relationships using correlation, to see what connects with what.
 - We'll also use some data basic analytics techniques.
 - Finally, implement K-Means clustering, and then evaluate the clustering performance using the F1 score, so we can judge how well it grouped things.
-

4. Methodology

4.1 Dataset Collection and Loading

The main dataset used here is the EPC Industry Analytics dataset. It was brought into Python with the Pandas library, and then keeping it inside a Data Frame, to work on it later. After loading, the data was explored, just to know how things were set up like the overall size of the data the column types, where the missing values showed up.

4.2 Data Exploration

Before jumping in, an initial scan of the dataset was conducted, to be aware of what's inside. The general shape of the data was checked, then looked at the data types for each column and the count of missing values across them. That step was useful because it pointed to possible data quality problems, things that might need some cleaning or preprocessing later, before any real analysis could start.

4.3 Duplicates

Pandas function was used to verify duplicates and remove it, if there were repeated entries in the dataset. The total count of duplicate rows was determined to evaluate data consistency.

4.4 Missing Values

Many of the missing values were detected during the data preparation phase. Empty fields, void strings, etc., were transformed into missing values. Numeric columns with missing values were populated using the median of their respective columns.

4.5 Data Analysis

Descriptive statistics was conducted on both numerical and categorical data. Metrics including count, average, minimum, maximum, etc., were produced to summarize the distribution of the data. A correlation analysis was performed to explore the relationships between numerical variables.

4.6 Detection of Outliers

To spot strange points, the Interquartile Range (IQR) approach was used, it is straightforward. At first, Q1 and Q3 was found for each numerical variable, and then computed the IQR. Every measurement was checked, if it landed outside the allowed interval, it got labeled as an outlier. Overall, this technique works well at catching those very high or very low values in the data, even when they look unrelated at first, or like they might belong but still don't quite fit.

4.7 Aggregation Based on Groups and Engineering of Features

Aggregation based on groups was conducted by categorizing records by categorical variables and determining summary statistics like mean, median, and total. A new feature was also developed by merging current numerical variables. This gave further understanding in data analysis.

4.8 Ranking, Conditional Selection.

Ranking analysis: It is assigning ranks to observations according to numerical values in descending order. Conditional filtering was used to find values exceeding the average of the chosen variable. These methods were used to show important points within the dataset.

4.9 Methodology (for machine learning)

Including the EPC data analysis, the MNIST digits dataset found in Scikit-learn was used for machine learning. The use of clustering was later measured with the F1 score.

4.10 Tools

- Python
- Google Colab
- Pandas
- NumPy
- Scikit-learn
- GitHub

4.11 GitHub (Repository)

<https://github.com/kirankumarkshastry/EPC-Industry-Analytics-Summer-2026>

5. Data Analysis and Results

5.1 Dataset Overview

The EPC Industry Analytics dataset was loaded into Python with the Pandas library and it went smoothly. At first, some exploration was conducted just to see how it's put together. In total the dataset has *1200 records and 14 columns*, and those columns mix numerical fields, categorical variables, and also some date related details. This first look gave a baseline understanding of the dataset before moving on to the preprocessing and the later analysis.

```
import pandas as pd
import numpy as np

df = pd.read_excel("EPC_Industry_Analytics_Demo.xlsx")

print("Shape:")
print(df.shape)

print("\nData Types:")
print(df.dtypes)

print("\nMissing Values:")
print(df.isnull().sum())
```

```
Shape:
(1200, 14)

Data Types:
Date                datetime64[ns]
Project_ID          object
Region              object
Project_Type         object
Contractor_Tier      object
Planned_Budget_K     float64
Actual_Cost_K        float64
Budget_Variance_Pct  float64
Sensor_Vibration_mm_s float64
Sensor_Temp_C        float64
Efficiency_Score     float64
Material_Price_Index float64
Unnamed: 12          float64
Unnamed: 13          object
dtype: object

Missing Values:
Date                0
Project_ID          0
Region              0
Project_Type        0
Contractor_Tier     60
Planned_Budget_K    0
Actual_Cost_K       0
Budget_Variance_Pct 0
Sensor_Vibration_mm_s 0
Sensor_Temp_C       0
Efficiency_Score    0
Material_Price_Index 0
Unnamed: 12         1200
Unnamed: 13         1199
dtype: int64
```

Figure 1: Initial exploration of the EPC dataset showing dataset structure & basic information

5.2 Analysis of Duplicate Records

Duplicate records were looked at to see if repeated observations were showing up in the dataset. From the check, it came out that there are 0 duplicate rows, so it seems the data is clear of duplicate entries and it can be used for additional analysis, no need for removing duplicates or any cleaning.

5.3 Handling Missing Values

Missing values were detected and addressed during the preprocessing phase. Blank entries, empty strings, and incorrect values were changed into missing values. Numerical columns with missing values were then filled using the median of each corresponding column. The dataset stayed together for additional analysis and reducing the effect of missing data.

5.4 Summary Statistics

A descriptive analysis was performed to capture the features of the dataset. Statistics including count, mean, median, minimum, maximum, etc., were calculated for numerical variables. Comparable summary data was gathered for categorical variables.

```
# Complete statistical summary
df.describe(include='all')
```

	Date	Project_ID	Region	Project_Type	Contractor_Tier	Planned_Budget_K	Actual_Cost_K	Budget_Variance_Pct	Sensor_Vibration_mm_s	Sensor_Temp_C	Efficiency_Score	Material_Price_Index	Unnamed: 12	Unnamed: 13
count	1200	1200	1200	1200	1140	1200.000000	1200.000000	1200.000000	1200.000000	1200.000000	1200.000000	1200.000000	0.0	1
unique	NaN	1200	4	4	3	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1
top	NaN	PROJ-2183	LATAM	Renewable Energy	Tier 2	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1
freq	NaN	1	317	314	574	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	1
mean	2023-05-30 12:00:00	NaN	NaN	NaN	NaN	2750.430575	2987.503387	8.482366	3.131770	54.708580	76.433536	110.917631	NaN	NaN
min	2023-01-01 00:00:00	NaN	NaN	NaN	NaN	500.052356	508.320284	-12.012275	0.045447	36.325602	0.000000	100.000000	NaN	NaN
25%	2023-03-16 18:00:00	NaN	NaN	NaN	NaN	1650.494262	1801.340050	3.374540	1.462936	47.091556	68.160112	105.460036	NaN	NaN
50%	2023-05-30 12:00:00	NaN	NaN	NaN	NaN	2751.711332	2967.673642	8.426886	2.473560	52.401493	80.128976	108.313578	NaN	NaN
75%	2023-08-13 06:00:00	NaN	NaN	NaN	NaN	3875.982343	4201.870148	13.529789	4.062142	60.197818	88.481085	118.146958	NaN	NaN
max	2023-10-27 00:00:00	NaN	NaN	NaN	NaN	4995.016754	6264.832318	29.201684	25.000000	112.475349	100.000000	121.113572	NaN	NaN
std	NaN	NaN	NaN	NaN	NaN	1298.446505	1431.969622	7.294449	2.868782	10.514297	16.656041	6.464250	NaN	NaN

Figure 2: Summary statistics.

5.5 Correlation

Correlation analysis was done to see how these numerical variables are connected across the dataset. When the correlation matrix is checked, it suggests a strong positive relationship between Planned_Budget_K and Actual_Cost_K. Most of the other variables were not highly correlated.

```
# Select numeric columns
numeric_df = df.select_dtypes(include=np.number)

# Correlation matrix
correlation_matrix = numeric_df.corr()

# Display result
print(correlation_matrix)
```

	Planned_Budget_K	Actual_Cost_K	Budget_Variance_Pct	\
Planned_Budget_K	1.000000	0.987637	0.039850	
Actual_Cost_K	0.987637	1.000000	0.181439	
Budget_Variance_Pct	0.039850	0.181439	1.000000	
Sensor_Vibration_mm_s	0.047613	0.040364	0.016270	
Sensor_Temp_C	-0.006899	-0.010928	-0.006320	
Efficiency_Score	0.019985	0.023992	0.006594	
Material_Price_Index	-0.000661	-0.007159	-0.035049	
Unnamed: 12	NaN	NaN	NaN	
	Sensor_Vibration_mm_s	Sensor_Temp_C	Efficiency_Score	\
Planned_Budget_K	0.047613	-0.006899	0.019985	
Actual_Cost_K	0.040364	-0.010928	0.023992	
Budget_Variance_Pct	0.016270	-0.006320	0.006594	
Sensor_Vibration_mm_s	1.000000	0.707830	-0.697970	
Sensor_Temp_C	0.707830	1.000000	-0.963516	
Efficiency_Score	-0.697970	-0.963516	1.000000	
Material_Price_Index	0.013068	0.021116	-0.012720	
Unnamed: 12	NaN	NaN	NaN	
	Material_Price_Index	Unnamed: 12		
Planned_Budget_K	-0.000661	NaN		
Actual_Cost_K	-0.007159	NaN		
Budget_Variance_Pct	-0.035049	NaN		
Sensor_Vibration_mm_s	0.013068	NaN		
Sensor_Temp_C	0.021116	NaN		
Efficiency_Score	-0.012720	NaN		
Material_Price_Index	1.000000	NaN		
Unnamed: 12	NaN	NaN		

Figure 3: Matrix illustrating the correlation of numerical variables in the EPC dataset.

5.6 Outlier Detection

Outlier detection was done using the Interquartile Range (IQR) approach. In the analysis, outliers were spotted across several variables, including Sensor_Vibration_mm_s, Sensor_Temp_C and Efficiency_Score. Those readings stayed in the dataset for analytical reasons, so they were not removed, and they are also mentioned alongside the general dataset characteristics.

```

# Select numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns

# Dictionary to store outlier counts
outlier_summary = {}

# Loop through each numeric column
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    outlier_summary[col] = len(outliers)

# Print dictionary
print(outlier_summary)

{'Planned_Budget_K': 0, 'Actual_Cost_K': 0, 'Budget_Variance_Pct': 3, 'Sensor_Vibration_mm_s': 41, 'Sensor_Temp_C': 32, 'Efficiency_Score': 33, 'Material_Price_Index': 0, 'Unnamed: 12': 0}

```

Figure 4: The number of outliers detected through the IQR method.

5.7 Group based aggregation

For the group-based aggregation, the data-set was grouped using one categorical variable, and ran summary stats for the numerical fields like the mean, the median, and the sum. This method made it easier to compare the numerical characteristics across categories and it gave a clearer picture of how project related variables changed between the group

```

# Find the first categorical (object) column
cat_col = df.select_dtypes(include='object').columns[0]

# Select numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns

# Group and aggregate
grouped_result = df.groupby(cat_col)[numeric_cols].agg(['mean', 'median', 'sum'])

# Display first 5 rows
print(grouped_result.head())

```

Project_ID	Planned_Budget_K mean	median	sum	Actual_Cost_K mean
PROJ-1000	2032.718750	2032.718750	2032.718750	2095.845956
PROJ-1001	822.703500	822.703500	822.703500	839.771030
PROJ-1002	2343.330027	2343.330027	2343.330027	2501.768017
PROJ-1003	1900.478728	1900.478728	1900.478728	1966.592256
PROJ-1004	3547.042879	3547.042879	3547.042879	3508.019596

Project_ID	median	sum	Budget_Variance_Pct mean	median	sum
PROJ-1000	2095.845956	2095.845956	3.105555	3.105555	3.105555
PROJ-1001	839.771030	839.771030	2.074566	2.074566	2.074566
PROJ-1002	2501.768017	2501.768017	6.761232	6.761232	6.761232
PROJ-1003	1966.592256	1966.592256	3.478783	3.478783	3.478783
PROJ-1004	3508.019596	3508.019596	-1.100164	-1.100164	-1.100164

Project_ID	Sensor_Vibration_mm_s mean	...	Sensor_Temp_C mean	Efficiency_Score mean
PROJ-1000	4.775256	...	63.900729	64.144031
PROJ-1001	1.779238	...	46.502858	83.031245
PROJ-1002	2.303336	...	50.985558	81.513558
PROJ-1003	3.341599	...	59.448200	72.523566
PROJ-1004	3.501433	...	59.905765	70.703579

Project_ID	median	sum	Material_Price_Index mean	median	sum
PROJ-1000	64.144031	64.144031	100.000000	100.000000	100.000000
PROJ-1001	83.031245	83.031245	100.058382	100.058382	100.058382
PROJ-1002	81.513558	81.513558	100.116760	100.116760	100.116760
PROJ-1003	72.523566	72.523566	100.175133	100.175133	100.175133
PROJ-1004	70.703579	70.703579	100.233497	100.233497	100.233497

Project_ID	mean	median	sum
PROJ-1000	NaN	NaN	0.0
PROJ-1001	NaN	NaN	0.0
PROJ-1002	NaN	NaN	0.0
PROJ-1003	NaN	NaN	0.0
PROJ-1004	NaN	NaN	0.0

[5 rows x 24 columns]

Figure 5: Group based aggregation results, showing summary statistics across categories.

5.8 Constructing Features

Feature engineering was performed by generating a new variable called New_Feature from the existing numerical variables. The new feature was created by dividing the initial numerical column by the second numerical column plus one. This activity showed how extra analytical variables can be obtained from current data and utilized for further examination.

```

# Find the first two numeric columns
numeric_cols = df.select_dtypes(include=np.number).columns

first_col = numeric_cols[0]
second_col = numeric_cols[1]

# Create new feature
df['New_Feature'] = df[first_col] / (df[second_col] + 1)

# Display first 5 rows
print(df.head())

```

```

...      Date Project_ID      Region      Project_Type Contractor_Tier \
0  2023-01-01  PROJ-1000  North America  Renewable Energy      Tier 3
1  2023-01-01  PROJ-1001      LATAM  Industrial Plant      Tier 1
2  2023-01-01  PROJ-1002      APAC    Oil & Gas      Tier 1
3  2023-01-01  PROJ-1003  North America  Industrial Plant      Tier 2
4  2023-01-02  PROJ-1004  North America  Renewable Energy      Tier 2

      Planned_Budget_K  Actual_Cost_K  Budget_Variance_Pct \
0      2032.718750      2095.845956      3.105555
1      822.703500      839.771030      2.074566
2     2343.330027     2501.768017      6.761232
3     1900.478728     1966.592256      3.478783
4     3547.042879     3508.019596     -1.100164

      Sensor_Vibration_mm_s  Sensor_Temp_C  Efficiency_Score \
0           4.775256      63.900729      64.144031
1           1.779238      46.502858      83.031245
2           2.303336      50.985558      81.513558
3           3.341599      59.448200      72.523566
4           3.501433      59.905765      70.703579

      Material_Price_Index  Unnamed: 12  Unnamed: 13  New_Feature
0           100.000000      NaN      NaN      0.969417
1           100.058382      NaN      NaN      0.978511
2           100.116760      NaN      NaN      0.936295
3           100.175133      NaN      NaN      0.965891
4           100.233497      NaN      NaN      1.010836

```

Figure 6: Newly developed attribute created from current numerical inputs.

5.9 Analysis of Rankings

A ranking analysis was conducted by allocating ranks to observations according to the values of a chosen numerical variable in descending sequence. Larger values obtained superior ranks, facilitating the identification of observations with relatively high numerical values and allowing for comparisons across the dataset.


```
# Select the first numeric column
first_numeric_col = df.select_dtypes(include=np.number).columns[0]

# Create rank column (highest value gets rank 1)
df['Rank'] = df[first_numeric_col].rank(ascending=False)

# Display first 5 rows
print(df[[first_numeric_col, 'Rank']].head())
```

	Planned_Budget_K	Rank
0	2032.718750	791.0
1	822.703500	1104.0
2	2343.330027	702.0
3	1900.478728	823.0
4	3547.042879	390.0

Figure 7: Order of observations according to numerical values.

5.10 Conditional Filtering

A conditional filtering approach was used, basically to pick out observations where the values were higher than the mean of the numerical variable were selected. So, the dataset that only contains above-average items. After cleaning, the dataset ended up with 602 rows and 16 columns, which means those rows and columns actually satisfy the set condition.

```
# First numeric column
first_numeric_col = df.select_dtypes(include=np.number).columns[0]

# Mean of that column
mean_value = df[first_numeric_col].mean()

# Filter rows greater than mean
filtered_df = df[df[first_numeric_col] > mean_value]

# Print shape
print(filtered_df.shape)
```

```
... (602, 16)
```

Figure 8: Outcomes of conditional filtering guided by the mean value standard.

5.11 K-Means Grouping on the MNIST dataset

The MNIST digits dataset was inserted using Scikit-learn and afterwards used for machine learning trials in essentially the same way. In this dataset, handwritten digit pictures get turned into numerical features at first, so the method can actually handle the information properly. After that, K-Means clustering was done with 10 clusters. The idea is that these 10 groups roughly line up with the digit classes 0,1,2... all the way to 9. Basically, this implies the algorithm works as if comparable patterns will be stuck together in the same local area or neighborhood. Essentially, the clustering process arranged similar digit shapes based on their shared characteristics.

```

from sklearn.cluster import KMeans

# Create K-Means model with 10 clusters
kmeans = KMeans(n_clusters=10, random_state=42)

# Fit model and get cluster labels
kmeans_labels = kmeans.fit_predict(X)

# Display first 10 labels
print(kmeans_labels[:10])

```

```

... [5 7 7 6 4 9 2 0 8 9]

```

Figure 9: Example output displaying cluster labels generated by the K-Means algorithm.

5.12 Clustering evaluation

K-Means clustering produces random cluster labels. The expected cluster labels were associated with the most common true labels in each cluster. The clustering model's performance was later assessed using F1 score. The achieved final F1 score was roughly 0.863, demonstrating effective clustering results for an unsupervised learning method.

```

from sklearn.metrics import f1_score
import numpy as np

# Map each cluster to the most common true label
mapped_labels = np.zeros_like(kmeans_labels)

for cluster in np.unique(kmeans_labels):
    mask = (kmeans_labels == cluster)
    true_labels = y[mask]

    most_common_label = np.bincount(true_labels).argmax()
    mapped_labels[mask] = most_common_label

# Calculate F1 score
f1 = f1_score(y, mapped_labels, average='macro')

print("F1 Score:", f1)

```

```

F1 Score: 0.8627854786352394

```

Figure 10: F1 score achieved after aligning cluster labels with actual digit labels.

5.13 Key Findings

The major findings of the project are:

- The EPC dataset had 1200 records and 14 columns.
- No duplicate records were detected in the dataset.
- Missing values were successfully identified and handled using median imputation.
- Strong relation between Planned_Budget_K and Actual_Cost_K.
- Outliers were present in variables (such as Sensor_Vibration_mm_s, Efficiency_Score, etc.).
- Conditional filtering made a dataset which has 602 rows and 16 columns

- Ranking, and conditional filtering provided additional analytical information.
 - K-Means clustering got a F1 score (around ~ 0.863).
-

6. Conclusion

This project provides direct practice with Exploratory Data Analysis, data cleaning, and machine learning in Python. The EPC Industry Analytics dataset went through this, like finding duplicate entries and verifying if it was, or wasn't, really duplicates, fixing missing values, doing statistical checks, checking correlations, spotting outliers, then grouping and aggregating by categories.

Basically, the dataset had no duplicate records at all, and that missing values were handled well using median. It also shows a strong positive relationship between the intended project budgets and the actual project expenses. Also, the project also used a machine learning approach with K-Means clustering on the MNIST digits dataset. This clustering produced an F1 score near 0.863, which tells that the clustering worked well for that kind of strategy.

Overall, this internship sharpens a student's understanding of Python powered data analytics, exploratory data analysis methods, and the basics of machine learning.

7. Appendices

Appendix A: References

Resources used:

1. Pandas - <https://pandas.pydata.org/docs/>
2. NumPy - <https://numpy.org/doc/>
3. Scikit-learn - <https://scikit-learn.org/stable/>
4. Google Colab - <https://colab.research.google.com/>
5. IDEAS TIH Summer Internship 2026 Project Notebook and Dataset

Appendix B: GitHub Repository

GitHub Repository:

<https://github.com/kirankumarkshastry/EPC-Industry-Analytics-Summer-2026>

The repository contains:

- Project notebook
 - Project report
 - Project files
-